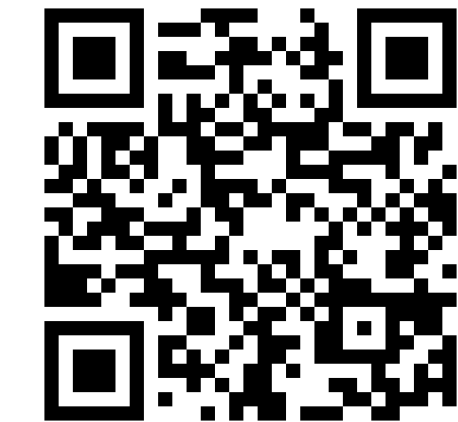


Agentic AI for Earth Science Software: a field report from building Worldscope

David Matthew Hall · NVIDIA · Berlin, Germany

An interactive Earth-system workbench. The AI agents in it are first-class users of every capability, with the same access a person has.



See it run
Source & walkthrough video

WHAT IT DOES

Built in 4½ months. Every commit written by an AI agent.

An **open-source Earth-2 tool** for weather, climate, and Earth-observation data, built to lower the barrier to entry for people who think visually. It runs on any device and uses whatever NVIDIA hardware is present for data services and AI inference. On a single desktop **DGX Spark** it runs the vast majority of AI weather models locally. It complements **Earth-2 Command Center**, which remains the right tool for photorealistic visualization and Omniverse digital twins.

DATA, FILES & FORMATS

- Reads **NetCDF**, **GRIB2**, **Zarr**, **GeoTIFF** across many grid types, including **HEALPix rendered natively** rather than interpolated to lat–lon first
- Opens remote files, and moves whole files or single fields between machines
- Many online sources: weather, climate, satellite observation
- Active weather caching, so animation plays at speed
- Every display carries units and a legend

MODELS & LIVE EARTH

- Launches **Earth2Studio** forecast, diagnostic, and assimilation runs from a GUI
- Ensemble member, vertical level, and time are all selectable axes
- Live conditions with animated winds and currents from several sources
- Live feeds: aircraft, ships, earthquakes, satellites
- Meteograms at a point, and side-by-side comparison through a swipe window

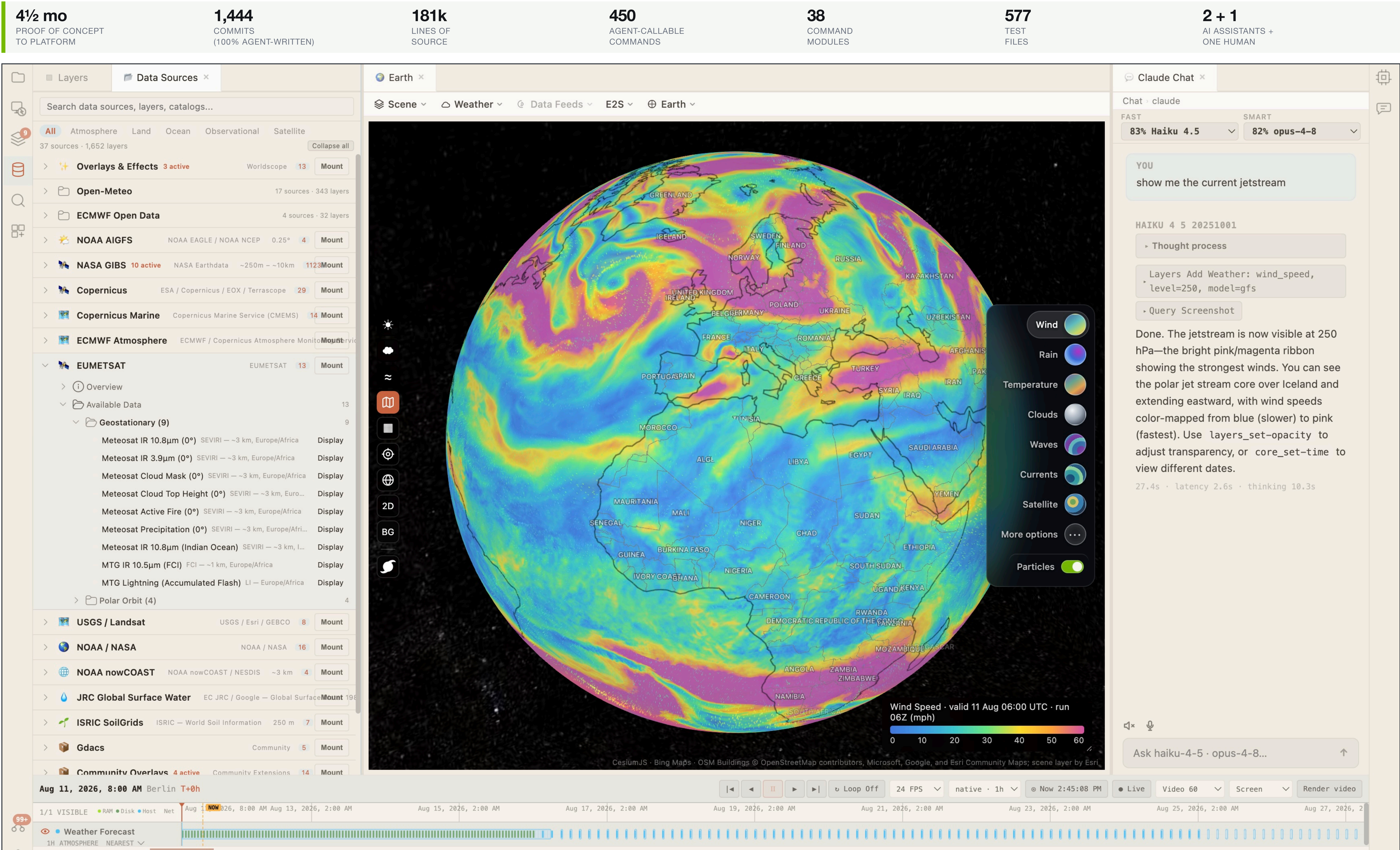
VIEWS & OUTPUT

- 2D and 3D at every scale, globe down to city blocks with buildings
- Sun, Moon, Mars, and other bodies for planetary missions
- Exports images, PDFs, and animations at chosen format and frame rate
- Multi-window, multi-tab shell modeled on VS Code, with a demo mode that hides the UI
- iPhone and iPad touch support in development

AGENTS AS USERS

- Every capability sits behind an **MCP server**. Agents call the same 450 commands the UI calls
- An agent loads data, runs a model, compares results, and drives the same scene the human sees
- **Several agents can work one scene at once**
- AI-driven demos: a slideshow of features that stays fully interactive

NEXT weather radar · Jupyter notebooks · large ensembles and forecast tracks · vertical-profile analysis · Earth-observation workflows such as change detection · AI model chains running global forecast → regional impact → decision support



Worldscope. A global forecast field on the interactive globe, with the layer explorer at left and an agent session at right. Through the Model Context Protocol the agent drives the same scene the human sees, sharing its layers, its camera, and its time cursor.

What building it taught me

1 The bottleneck moved downstream.

For well-specified features, agents produced implementations roughly **a hundred times faster** than I could have written them. The cost of implementation collapsed. The effort moved into specification, integration, and verification, none of which got cheaper.

4 Tests caught regressions, not wrongness.

Agent-written tests found breakage, but **could not establish that a complex interactive workflow behaved correctly end to end**. Code that is plausible, runnable, and wrong is the dangerous case in scientific software, and a green test suite does not catch it.

2 Holding the whole thing together got harder.

As implementation got cheaper, keeping a coherent whole mattered **more**. Left to themselves the agents converged on the ordinary layout, the common grid, the default colormap. They reach for whatever already exists somewhere in the training data.

5 Running agents in parallel had a cost.

It accelerated implementation and multiplied conflicting assumptions and integration regressions at the same time. The tools were also changing under me through the whole build, and each new assistant arrived with different strengths and its own odd habits.

I served as architect, tester, and director. The agents wrote the code. What follows is how that felt from the inside, and the questions it raises for everyone here. None of them are specific to this project.

3 Verification was the hardest part.

Interactive behavior was the worst of it, for a structural reason. The agents observed the browser through **still screenshots** and had to infer what happened in between. Features often looked correct and did not work. Fixing one broke another without warning.

Where the effort went

Schematic. Impressions from one project, not measurements.

WRITING IT MYSELF

Implementation	Spec	Integ.	Verify
----------------	------	--------	--------

DIRECTING AGENTS

·	Specification	Integration	Verification
■ Writing code	■ Saying what it must do	■ Making the parts fit	■ Proving it is right

OPEN QUESTIONS FOR THIS COMMUNITY

Answers welcome. Arguments preferred.

I WHERE THE WORK GOES

- What happens when AI agents become *full participants* in an Earth-science workflow instead of chatbots attached to one?
- If writing code becomes a hundred times cheaper, which part of the scientific workflow becomes the bottleneck, and is any group staffed for it?
- Is the durable artifact still the source code, or the *specification* that regenerates it?
- What is scientific programming a skill *in*, once the programming is free?

IV PROVENANCE & REVIEW

- If a published result rests on 180,000 lines no human has read line by line, what exactly was peer-reviewed?
- What should journals and programme committees require? Prompts, model versions, tool versions, transcripts, seeds?
- Is agent-generated code reproducible once the agent that wrote it has been deprecated?
- Who is the author of a scientific tool, and what counts as a citable software contribution now?

II VERIFICATION & PHYSICALITY

- How do you verify software you did not write, at the rate it is now written?
- Who catches code that runs, passes its tests, and is *physically* wrong? Flipped signs, unit slips, staggered-grid offsets, silent regridding, mishandled missing values, the wrong CRS.
- Generation scales and checking does not. Is that asymmetry the ceiling on agentic science?
- What is the software equivalent of a physicality check? Conservation, invariance, dimensional consistency, known-answer tests?

V THE GENERIC ATTRACTOR

- Agents regress toward the mean of their training data. Does that pull the whole field toward common grids, common formats, and mainstream methods?
- How much of scientific novelty lives in the choices an agent would talk you out of?
- Does agentic development accelerate science, or accelerate its *consolidation*?
- Do we get more science, or the same science faster?

III EVALUATION & BENCHMARKS

- We benchmark the models. Should we benchmark the *code that produces them*, and what would that benchmark measure?
- Agents perceive interactive systems as still frames and infer the motion in between. What does that imply for evaluating anything time-dependent, interactive, or visual?
- Do agent-built pipelines alter our datasets by resampling, gap-filling, or reprojecting in ways our metrics never see?
- If an agent writes the model, the training loop, *and* the evaluation, what independence is left?

VI PEOPLE, ACCESS, COMMONS

- If early-career scientists never write the code, where does the judgment to review it come from, and what do we train them to be good at?
- Does this widen the gap between well-resourced and under-resourced groups, or erase it?
- When a bespoke tool costs a weekend, do we still build shared community software, or does the toolchain fragment?
- Is scientific software still worth *maintaining*, or is it now disposable and regenerable?

As the cost of intelligence drops rapidly toward zero, the scarce resource turns out to be **artistic vision and judgment**.

If that is right:
are we selecting for it, teaching it,
or rewarding it?